

# NPTEL

## U-Net Architecture in DDPM

# Contents

- ✓ Role of U-Net in DDPM
- ✓ Time Embedding
- ✓ Encoder Block
- ✓ Bottleneck Block
- ✓ Decoder Block
- ✓ Residual Blocks
- ✓ Self-Attention
- ✓ Complete DDPM U-Net

# Role of U-Net in DDPM



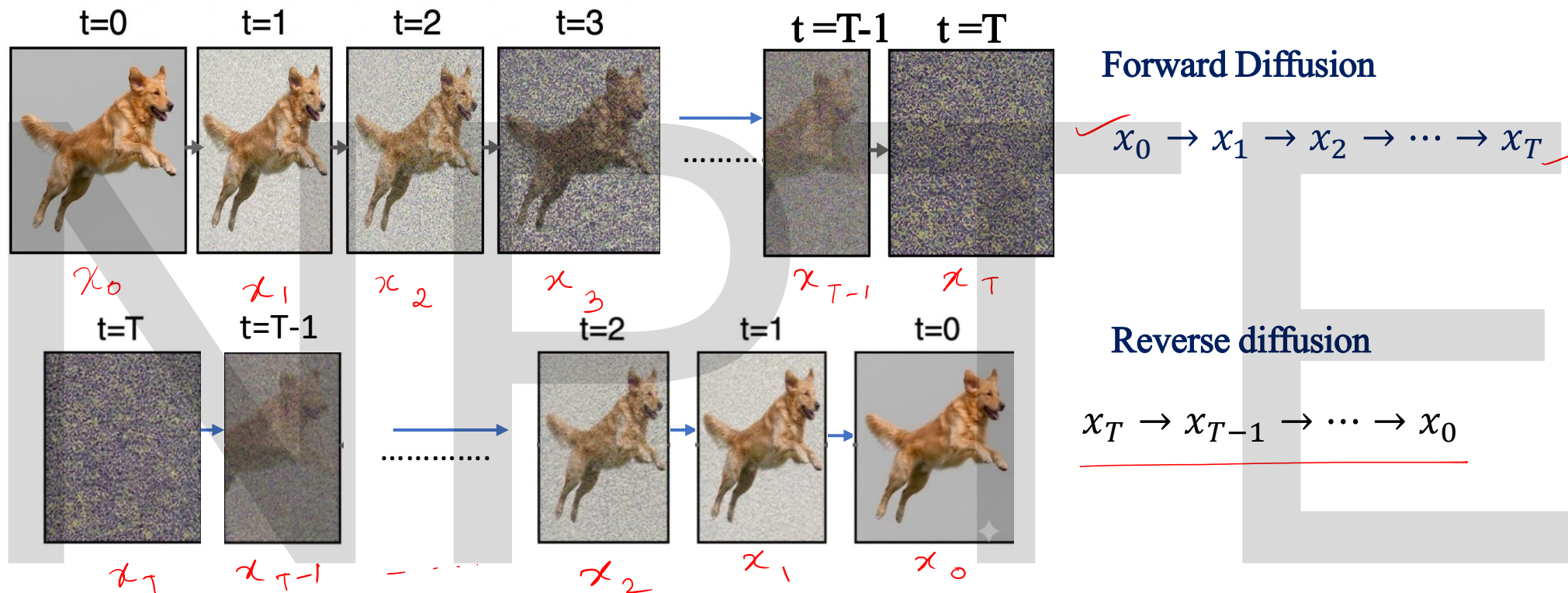
Main Task

It learns to predict the Gaussian noise  $\epsilon$  present in the noisy image  $x_t$  conditioned on timestep  $t$ .

Why is U-Net used?

- Captures global context (overall image structure)
- Preserves fine details via skip connections

# Role of U-Net in DDPM Pipeline



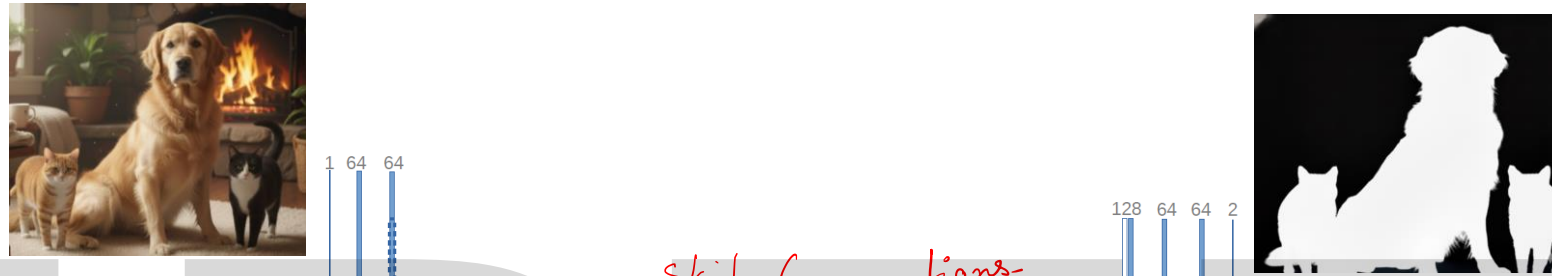
✓ At every reverse step:

is predicted by the U-Net.

$$\epsilon_{\theta}(x_t, t)$$

$$x_t \rightarrow \epsilon_{\theta}(x_t, t) \rightarrow x_{t-1}$$

# U-Net Architecture-Original



Encoder  
(Contracting path)

Decoder  
(Expanding path)

DDPM uses a modified U-Net with

- ✓ Timestep embeddings.
- ✓ Residual blocks and
- ✓ Attention layers

Skip Connections

output  
segmentation  
map

- conv 3x3, ReLU
- copy and crop
- ↓ max pool 2x2
- ↑ up-conv 2x2
- conv 1x1

Bottleneck

# Time Embedding in DDPM U-Net

- ✓ Time embedding encodes the diffusion timestep  $t$ , which indirectly indicates the noise level.
- ✓ Without it, the network would see only  $x_t$  and would not know whether it came from an early diffusion step (little noise) or a late step (almost pure noise).

$t = 50 \rightarrow$  slightly noisy image ✓  
 $t = 500 \rightarrow$  moderately noisy image ✓  
 $t = 900 \rightarrow$  heavily noisy image ✓

At each training step, DDPM provides:

$(x_t, t)$  ✓

where:

$x_t$  = noisy image

$t$  = diffusion timestep

$x_T \rightarrow x_{T-1} \dots x_t \dots x_2 \rightarrow x_1 \rightarrow x_0$   
timestep  
 $= t$

# Convert Timestep into a Time Embedding Vector

Instead of feeding the scalar  $t$  directly, DDPM converts it into a high-dimensional vector using sinusoidal embeddings (similar to Transformers).

For embedding dimension  $d$

$$\text{TE}(t)_{2i} = \sin\left(\frac{t}{10000^{2i/d}}\right)$$

$$\text{TE}(t)_{2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

Typical implementations use embedding dimensions between 128 and 512.

Each  $i$  gives 2 terms:

- ✓ One sine term
- ✓ One cos term

$$i = 0, 1, 2, \dots$$

Suppose  $t = 100$   
 $\& d = 8$ .

For  $i = 0$

$$(10000)^{2i/d} = (10000)^{(2 \times 0)/8} = (10000)^0 = 1$$

argument:  $-\frac{100}{1} = 100$

$\left\{ \frac{t}{10000} \right\}^{2i/d}$

$$TE)_{2i} = TE_0 = \sin(100) \approx -0.506$$

$$TE)_{2i+1} = TE_1 = \cos(100) \approx 0.862$$

For  $i = 1$

$$\frac{2i}{d} = \frac{2 \times 1}{8} = \frac{10000^{0.25}}{10000} = 10000^{0.25} = 10$$

argument:  $-\frac{100}{10} = 10$

$$TE)_{2 \times i} = TE)_{2 \times 1} = TE_2 = \sin(10) \approx -0.544$$

$$TE)_{2i+1} = TE_3 = \cos(10) \approx -0.839$$



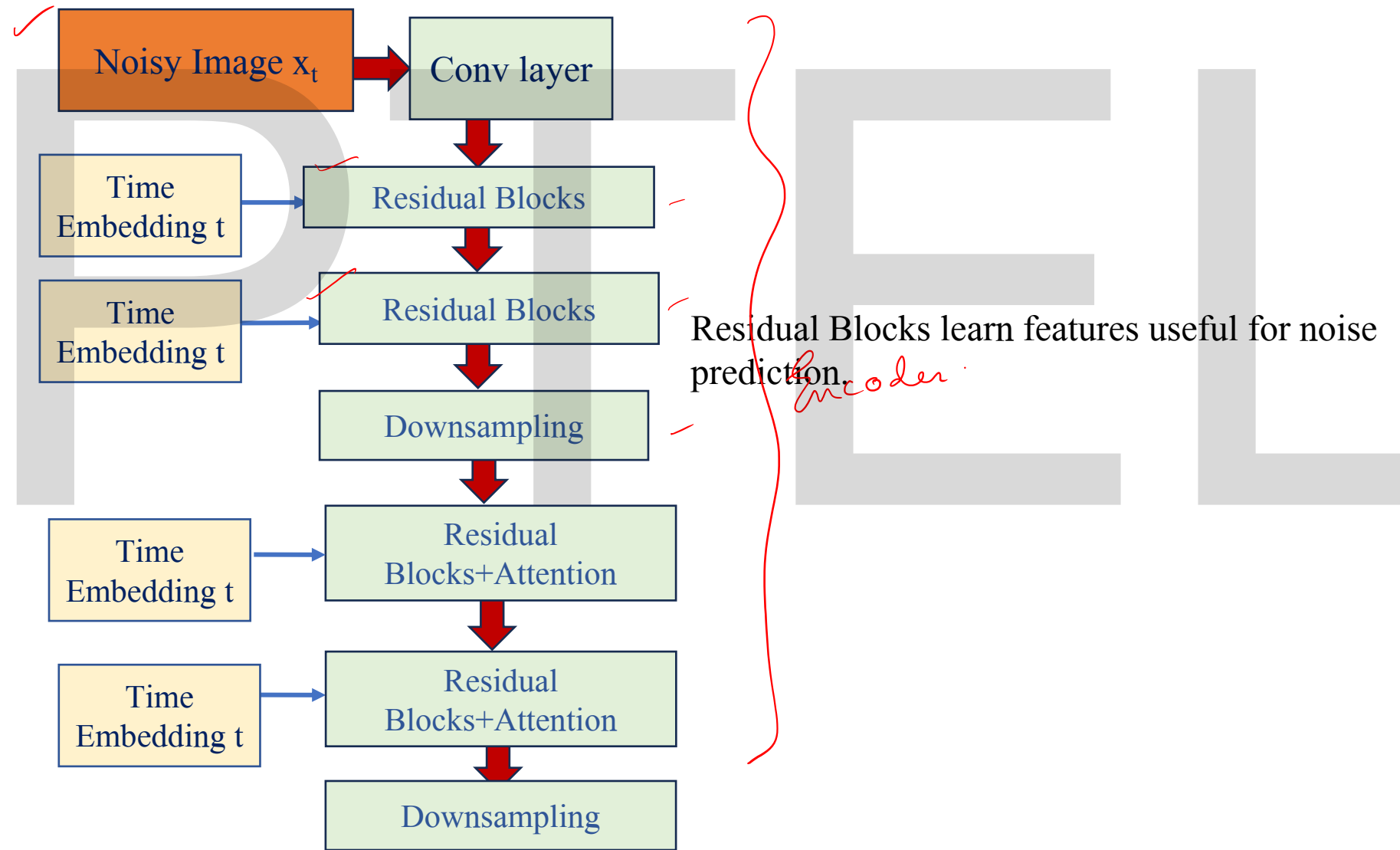
// by for  $i=2$  we get  $TE_4, TE_5$   
for  $i=3$  we get  $TE_6, TE_7$

$\therefore$  Total =  $TE_0, TE_1, TE_2, TE_3$  -----  $TE_7$ .

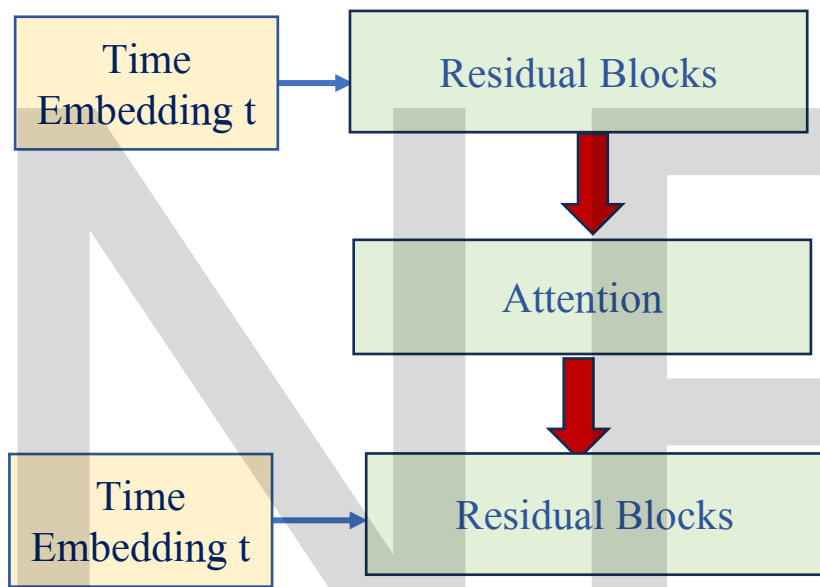
$\therefore TE(100) = \begin{bmatrix} -0.506, & 0.862 & -0.544 & -0.839 \\ & - & - & - \\ & - & - & - \end{bmatrix}$  8 dimensional vector which is fed to the U-net architecture.

# Encoder side of U-Net

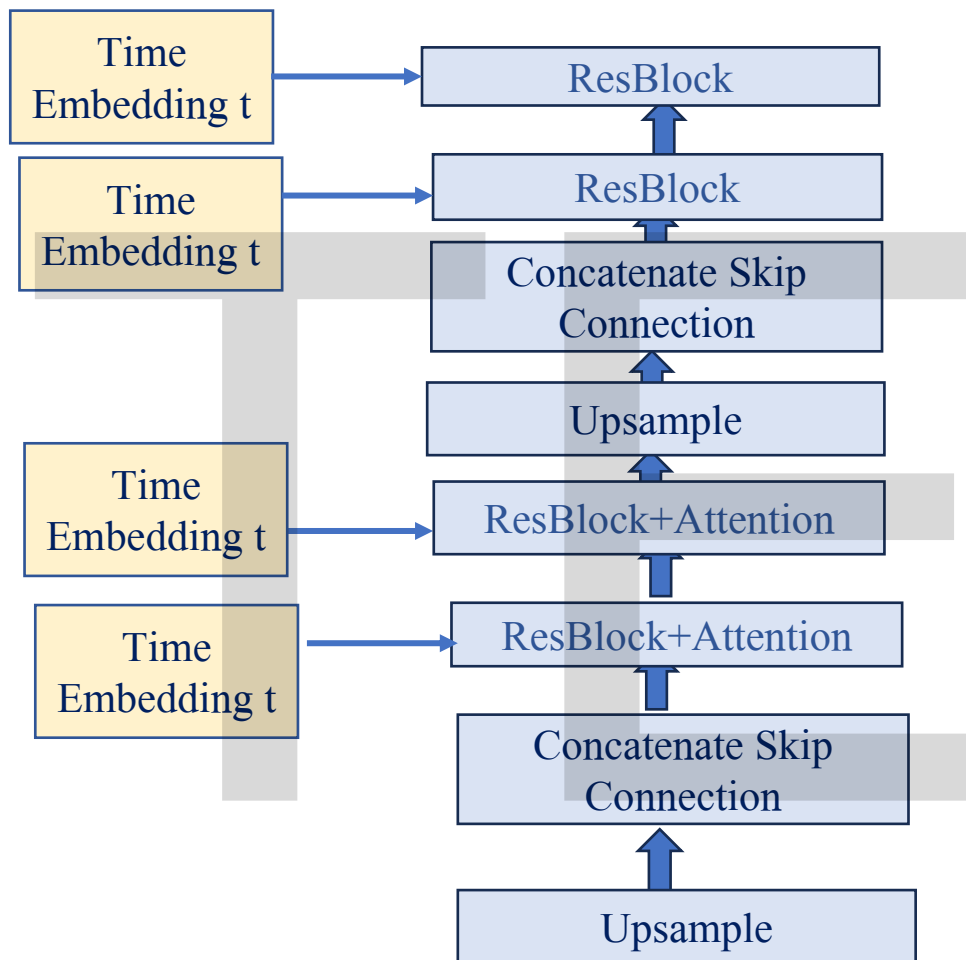
The encoder extracts increasingly abstract features.



## Bottleneck

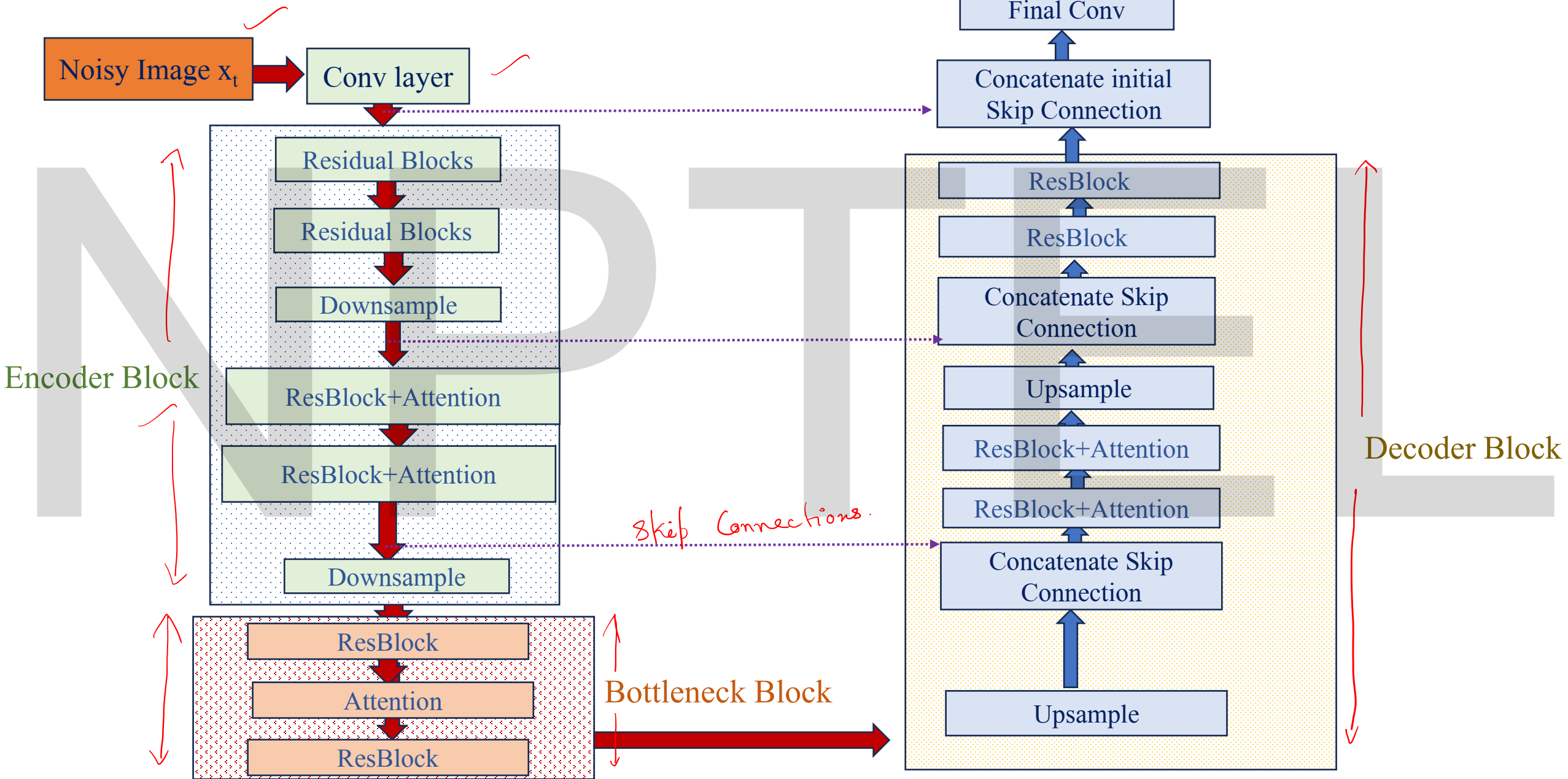


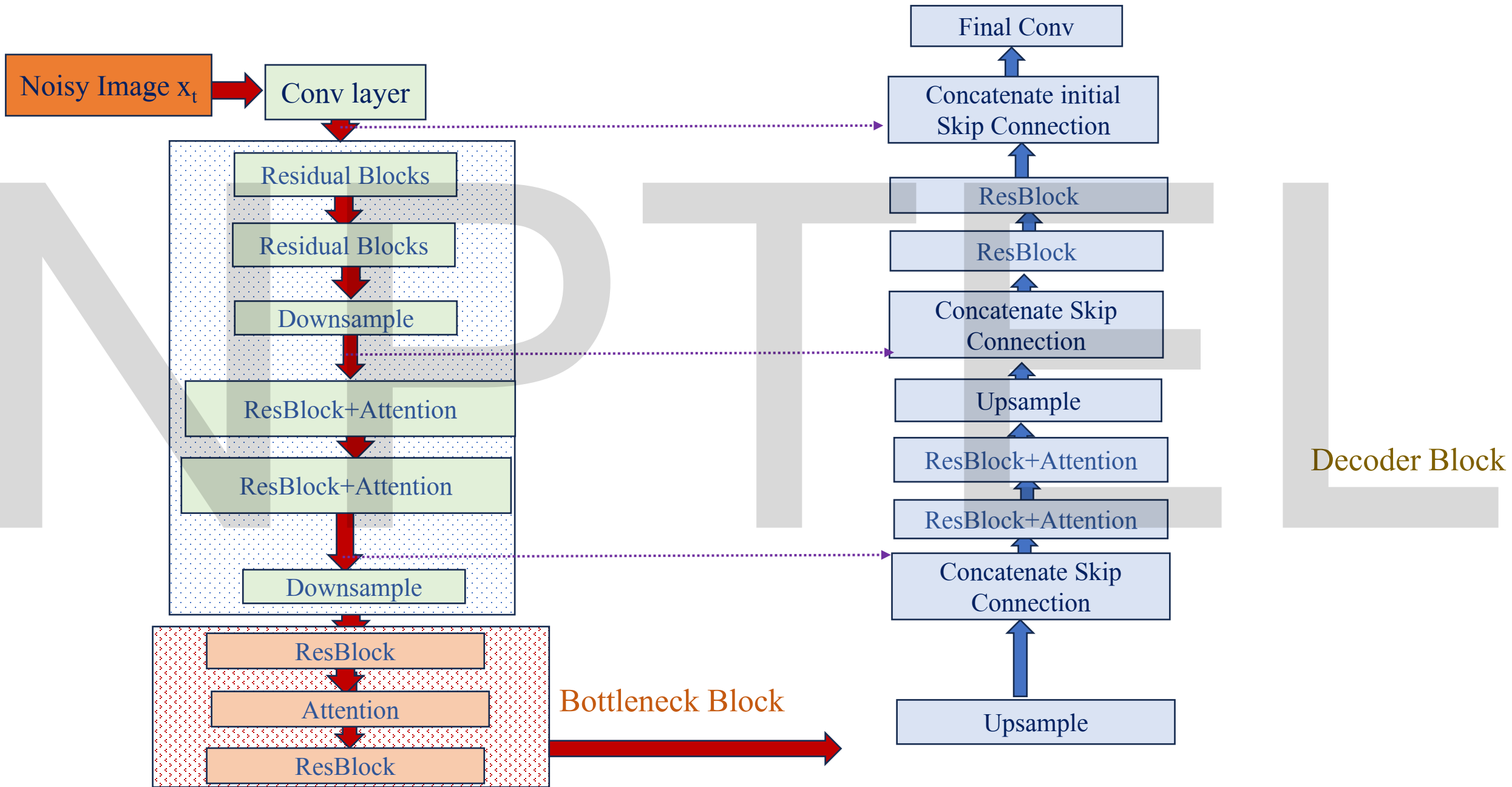
## Decoder Block



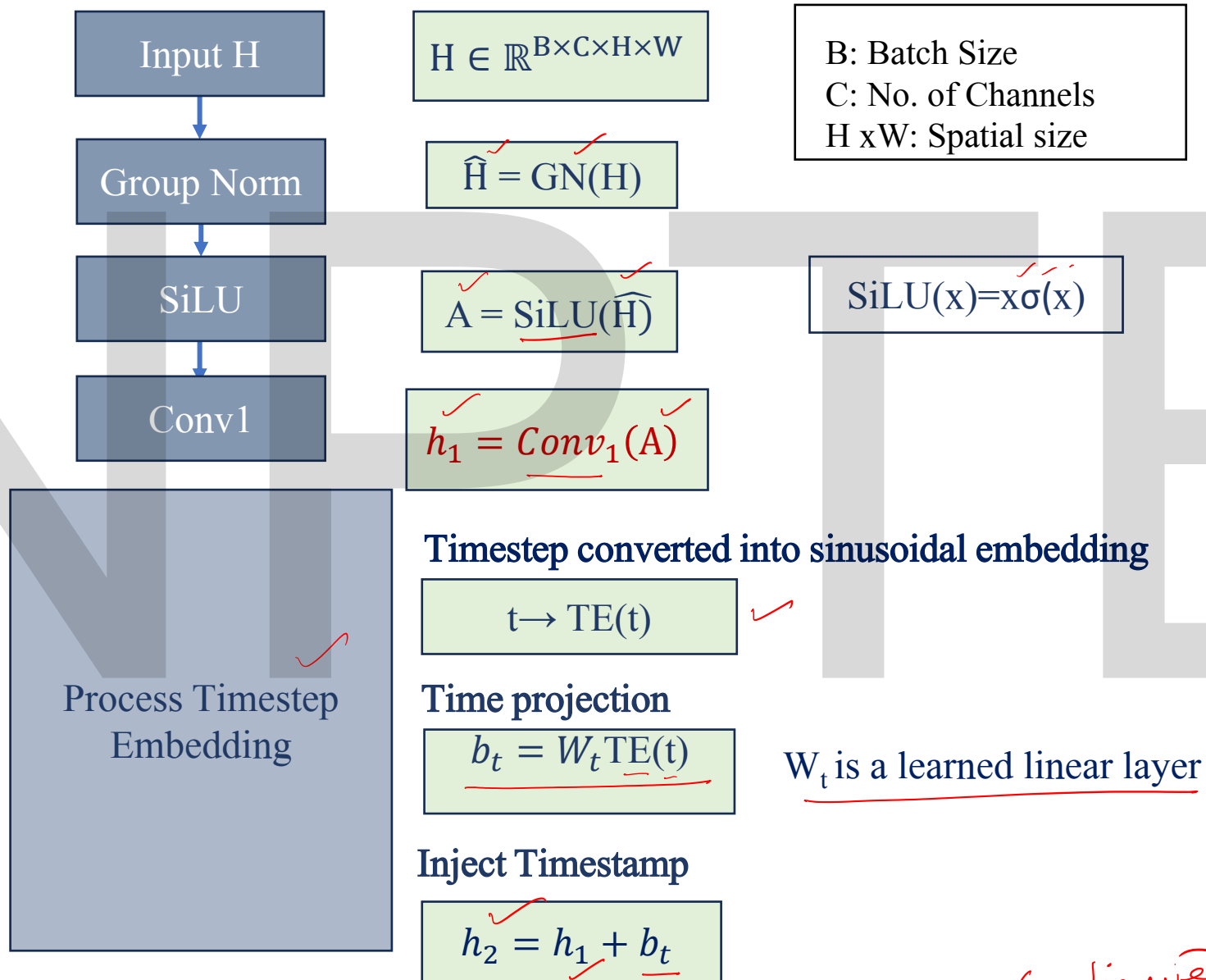
$x_t \rightarrow \text{Encoder} \rightarrow \text{Bottleneck} \rightarrow \text{Decoder} \rightarrow \epsilon \rightarrow \text{predicted noise.}$

# Simplified DDPM U-Net





# Inside the Residual Block



Continued

Process Timestep  
Embedding

Group Norm

$$h_3 = \text{GN}(h_2)$$

SiLU

$$h_4 = \text{SiLU}(h_3)$$

Conv2

$$h_5 = \text{Conv2}(h_4)$$

Residual  
Connection

$$H_{\text{out}} = H + h_5$$

Output

$F(H) = h_5$   
The block learns  
$$H_{\text{out}} = H + F(H)$$

$F(H)$  is the learned transformation

# Self-Attention in DDPM U-Net

- ✓ Convolutions see only a local neighborhood.
- ✓ A pixel may need information from distant regions of the image.
- ✓ Self-attention enables the network to capture **long-range dependencies**.

## Example

- ✓ To denoise the eye of a face, the network may need information from the other eye located far away in the image.

## How Self-Attention Works

Given a feature map:

Compute:

$Q$  = Query ,  $K$  = Key  $V$  = Value

$$X \in \mathbb{R}^{H \times W \times C}$$

$$\begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned}$$

Attention weights:

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

Output:  $Y = AV$



NPTEL

Summary

*Next Session*  
*Classifier Guided Diffusion*